

# Détection de Tweets Suspects

Approche Machine Learning avec gestion de versions Git & DVC

## INTRODUCTION

Les réseaux sociaux, et Twitter (X) en particulier, constituent aujourd'hui l'une des principales sources d'information instantanée, mais également l'un des principaux vecteurs de diffusion de contenus suspects : désinformation, discours haineux, spam, tentatives de manipulation ou de fraude.

L'identification automatique de ces contenus représente un enjeu majeur pour la modération des plateformes, la protection des utilisateurs et la lutte contre la désinformation.

Ce projet a pour objectif de construire une chaîne complète de **Machine Learning** permettant de classifier automatiquement un tweet comme *suspect* ou *non suspect*, en couvrant l'ensemble du cycle de vie d'un projet de Data Science : exploration des données, prétraitement du texte, représentation vectorielle, entraînement et comparaison de plusieurs algorithmes, gestion du déséquilibre des classes, optimisation des hyperparamètres, et enfin déploiement du modèle sous forme d'application interactive.

- Réaliser une analyse exploratoire approfondie des tweets fournis.
- Mettre en place un pipeline de prétraitement textuel robuste et documenté.
- Représenter les tweets sous forme numérique exploitable par des modèles de ML.
- Comparer plusieurs algorithmes de classification et gérer le déséquilibre des classes.
- Versionner les données et le pipeline avec **Git** et **DVC** pour garantir la reproductibilité.
- Optimiser le modèle retenu et l'évaluer selon des métriques rigoureuses.
- Déployer le modèle final sous forme d'application Streamlit interactive.

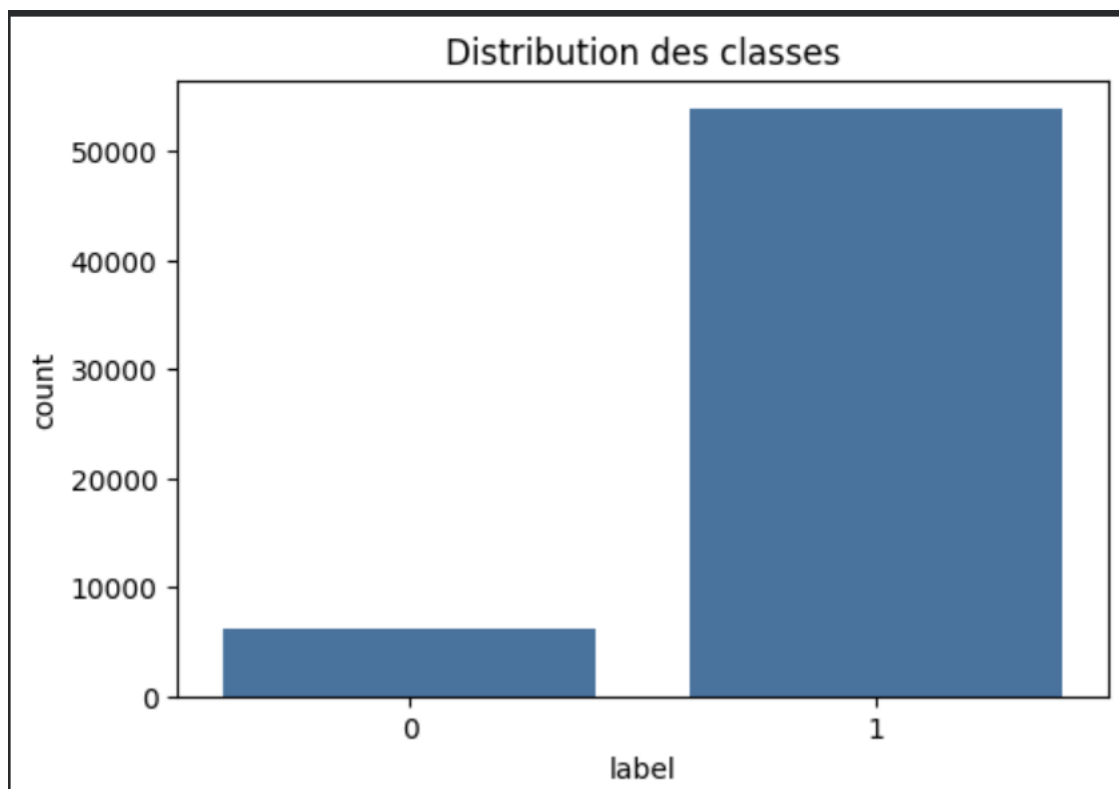
Ce rapport suit la structure du cahier des charges de l'examen : après une présentation du jeu de données et de son exploration, nous détaillons la méthodologie de prétraitement, la stratégie de gestion des données avec DVC, les choix de représentation vectorielle, la construction et la comparaison des modèles, les résultats obtenus, l'optimisation effectuée, puis le déploiement de la solution. Une section de discussion finale revient sur les limites du projet et les perspectives d'amélioration.

## Exploration et prétraitement des données

Le jeu de données est chargé à l'aide de la librairie Pandas à partir du fichier CSV fourni. Il contient, pour chaque observation, le texte brut du tweet ainsi qu'une étiquette binaire indiquant si le contenu est jugé suspect (1) ou non (0).

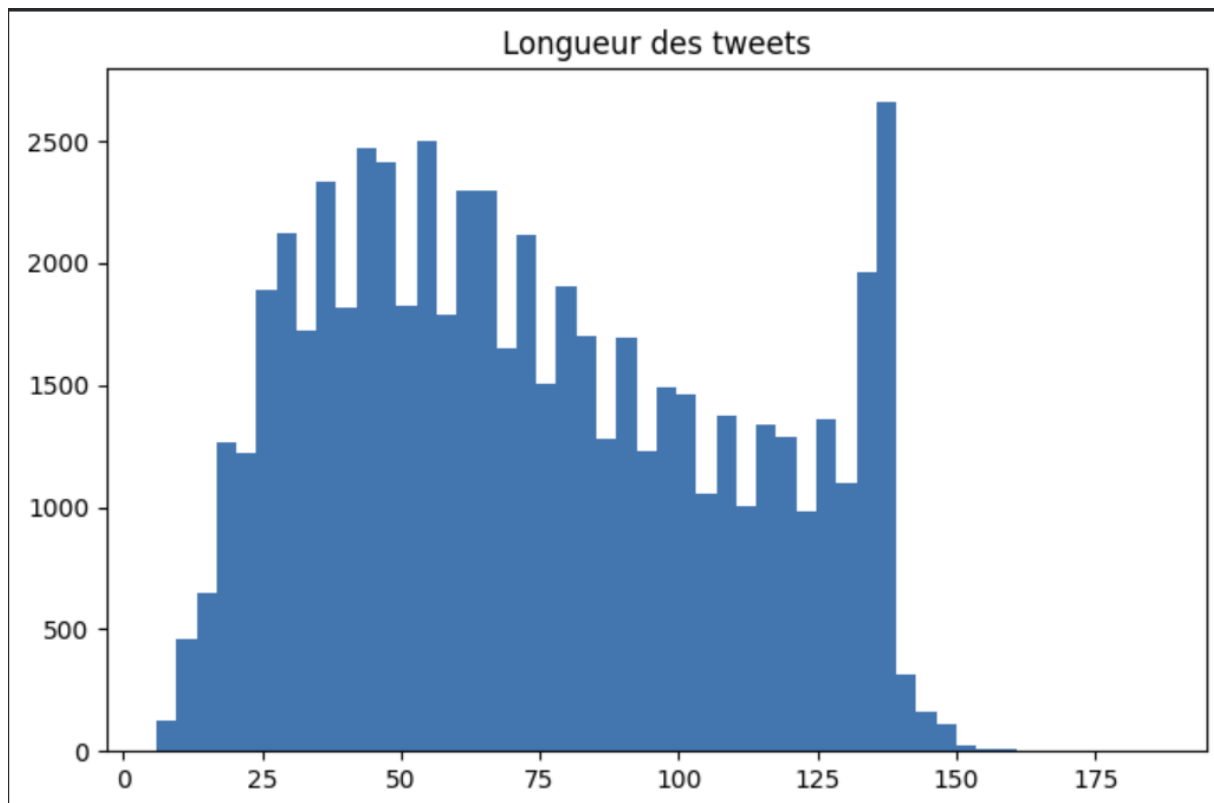
L'inspection initiale permet d'identifier les colonnes disponibles (identifiant du tweet, texte, label, et éventuellement des métadonnées telles que la date ou l'auteur), de vérifier l'absence de doublons, et de quantifier les valeurs manquantes. Les lignes avec un texte vide ou manquant sont supprimées, et les doublons exacts sont éliminés afin d'éviter toute fuite d'information entre les ensembles d'entraînement et de test.

L'analyse de la distribution des classes révèle un déséquilibre marqué : la classe « suspect » est nettement minoritaire par rapport à la classe « non suspect ». Ce constat oriente directement les choix méthodologiques de la partie 4 (gestion du déséquilibre).



La distribution du nombre de mots par tweet est également étudiée. On observe généralement que les tweets suspects ont tendance à être légèrement plus longs et contiennent davantage de mots en

D'autres visualisations complémentaires peuvent être produites : nuage de mots par classe, fréquence des hashtags, présence d'URLs, taux d'emojis, proportion de majuscules, etc. Ces analyses permettent de mieux cerner les signaux discriminants avant la phase de modélisation.



### Prétraitement du texte

Le nettoyage du texte est une étape critique pour réduire le bruit et la dimensionnalité du vocabulaire. Le pipeline de prétraitement retenu applique séquentiellement les opérations suivantes :

- **Conversion en minuscules** : uniformisation de la casse pour éviter la duplication de tokens.
- **Suppression des URLs** : les liens (http, https, t.co, www) n'apportent pas d'information sémantique directe et sont supprimés via une expression régulière.
- **Suppression des mentions et hashtags bruts** (le symbole est retiré, le mot-clé peut être conservé car souvent informatif).
- **Suppression des caractères spéciaux et de la ponctuation** non porteuse de sens.
- **Suppression des stop words** (mots vides : « le », « la », « et », « the », « is »...) à l'aide de NLTK ou spaCy selon la langue du corpus.
- **Lemmatisation** (ou stemming) pour ramener les mots à leur forme canonique et réduire la taille du vocabulaire.

## Gestion des données et du pipeline avec Git & DVC

Le projet est initialisé comme un dépôt Git classique, auquel est ajoutée la gestion DVC pour le suivi des données volumineuses et des artefacts de modèle, qui ne doivent pas être versionnés directement dans Git.

Cette approche garantit que seul un petit fichier, `dvc` (pointeur avec hash du contenu) est versionné dans Git, tandis que les données réelles sont stockées sur le remote configuré (Google Drive, S3 ou stockage local), ce qui permet à toute personne clonant le dépôt de récupérer les données via **dvc pull**.

Le pipeline est défini dans un fichier **dvc.yaml**, comprenant au minimum trois étapes : le prétraitement, l'entraînement et l'évaluation. Chaque étape déclare ses dépendances (scripts et données) et ses sorties, ce qui permet à DVC de ne ré-exécuter que les étapes impactées par un changement. Grâce à cette configuration, toute personne clonant le dépôt GitHub peut reproduire intégralement les résultats via les deux commandes suivantes :

```
dvc pull # récupère les données et artefacts versionnés
dvc repro # ré-exécute le pipeline si des dépendances ont changé
```

DVC détecte automatiquement, via le hash des fichiers, si une étape doit être relancée. Le fichier **dvc.lock** fige les versions exactes de chaque dépendance et sortie, assurant une traçabilité complète de l'expérimentation.

## **Représentation des données (Feature Engineering)**

Parmi les approches proposées (Bag of Words, TF-IDF, Word2Vec, GloVe, FastText, Sentence

Transformers, BERT), le choix s'est porté sur une représentation **TF-IDF** (Term Frequency – Inverse

Document Frequency) comme approche principale, complétée à titre de comparaison par des

**embeddings pré-entraînés (Word2Vec / Sentence-BERT)** pour les modèles les plus avancés.

### **Justification du choix**

- TF-IDF offre un excellent compromis **performance / simplicité / interprétabilité** pour des modèles classiques (Logistic Regression, Naive Bayes, SVM, Random Forest, XGBoost).
- Les tweets étant courts, les approches de type *Bag of Words* pur souffrent davantage de la dimensionnalité et du bruit ; TF-IDF pondère les termes rares et discriminants.
- Les Sentence Transformers (embeddings contextuels) sont testés en complément pour capturer la sémantique globale du tweet et sont utilisés notamment pour un modèle de type BERT fine-tuné.

## **Construction et comparaison des modèles**

```

=====
Logistic Regression
      precision    recall  f1-score   support

     0       0.58      0.79      0.67      1229
     1       0.97      0.93      0.95     10771

 accuracy      0.92      12000
 macro avg      0.78      0.86      0.81      12000
 weighted avg      0.93      0.92      0.92      12000

```

```

=====
Naive Bayes
      precision    recall  f1-score   support

     0       0.40      0.85      0.54      1229
     1       0.98      0.86      0.91     10771

 accuracy      0.85      12000
 macro avg      0.69      0.85      0.73      12000
 weighted avg      0.92      0.85      0.88      12000

```

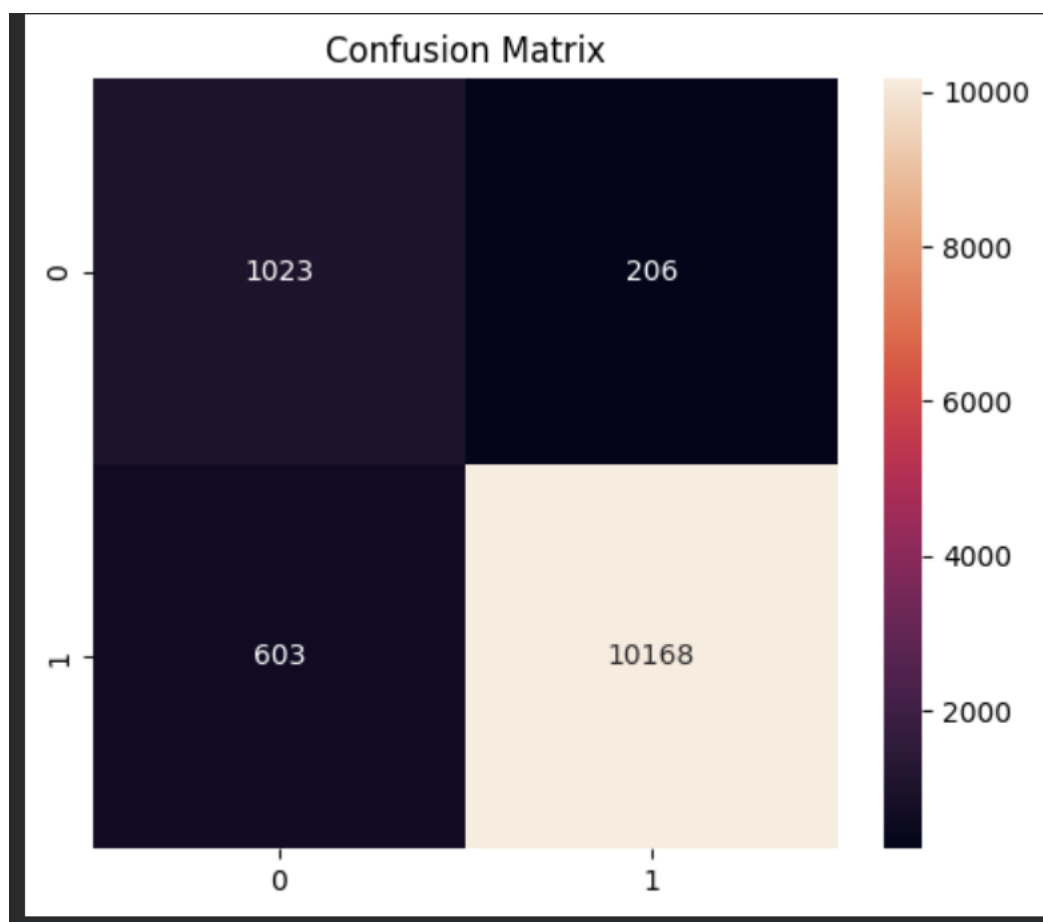
```

=====
SVM
      precision    recall  f1-score   support

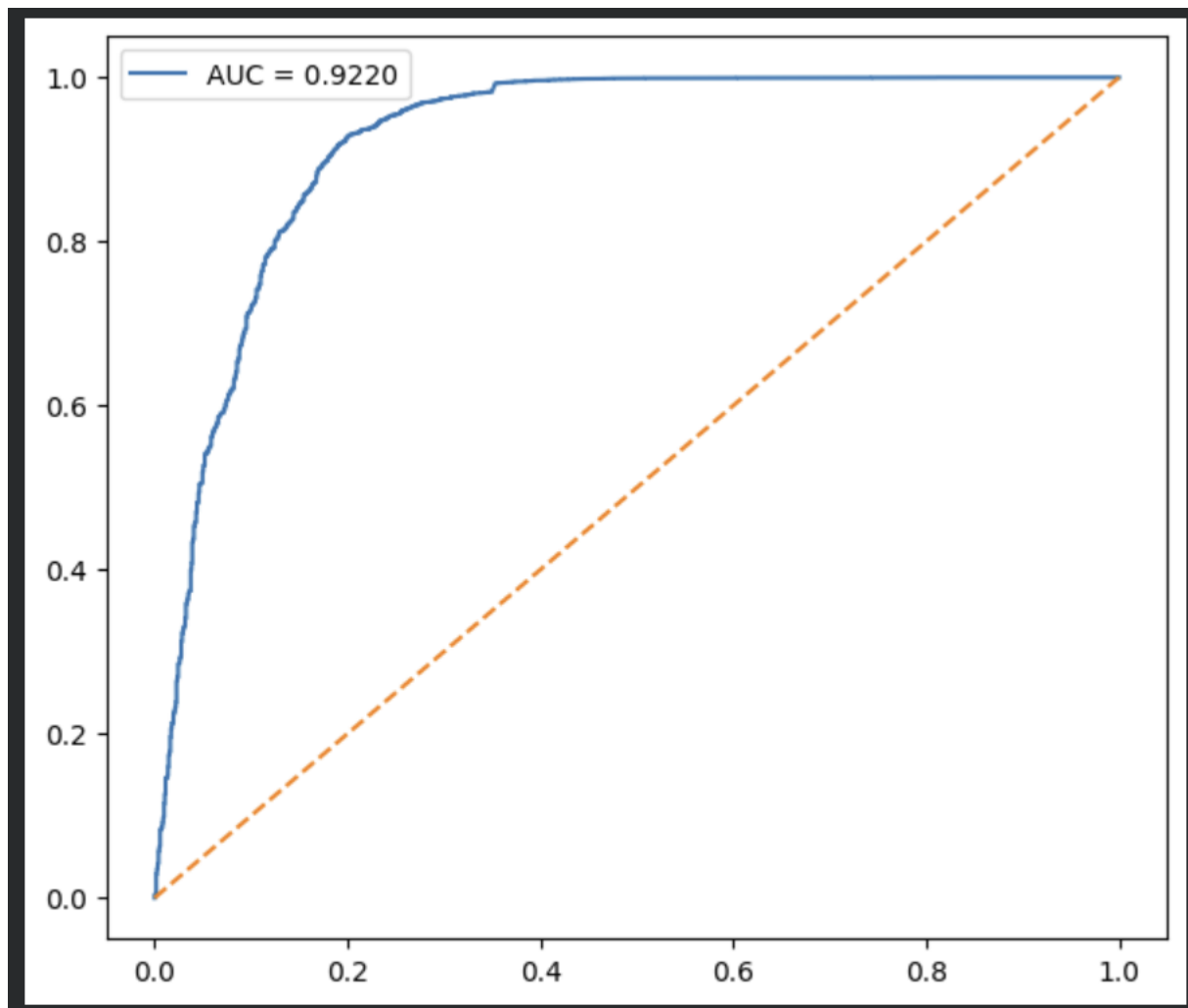
     0       0.63      0.83      0.72      1229
     1       0.98      0.94      0.96     10771

 accuracy      0.93      12000
 macro avg      0.80      0.89      0.84      12000
 weighted avg      0.94      0.93      0.94      12000

```



La courbe ROC



## Déploiement (Application Streamlit)

La solution retenue pour le déploiement est une **application Streamlit**, offrant une interface simple et interactive permettant à un utilisateur de saisir un tweet et d'obtenir instantanément la prédiction du modèle ainsi que la probabilité associée.

# Détection de Tweets Suspects

Saisissez un tweet pour savoir s'il est considéré comme suspect par le modèle.

Entrez un tweet

Hollis' death scene will hurt me severely to watch on film wry is directors cut not out now?

Prédire

## Résultat

Non suspect

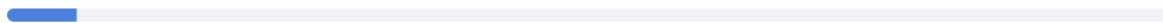
Classe prédite : 0

## Probabilités

Non suspect : 93.50%



Suspect : 6.50%



Confiance du modèle : 93.50%

## Conclusion

Ce projet a permis de mettre en oeuvre l'intégralité du cycle de vie d'un projet de Machine Learning appliqué à la détection de tweets suspects : de l'exploration et du nettoyage des données à leur représentation vectorielle, en passant par la comparaison rigoureuse de plusieurs algorithmes, la gestion du déséquilibre des classes, l'optimisation des hyperparamètres, jusqu'au déploiement d'une application interactive.

L'intégration de **Git** et **DVC** a permis d'assurer la traçabilité et la reproductibilité complète des

Expérimentations, un aspect essentiel des bonnes pratiques MLOps

Les perspectives identifiées (modèles Transformers, monitoring, CI/CD, MLflow) constituent des axes

Naturels d'évolution pour renforcer la robustesse et la pérennité de la solution en conditions réelles de production.